Project 2: Distance Vector Routing

Discussion Questions

1. Compare distance-vector and link-state routing in terms of stored information, control traffic, convergence, and implementation complexity.

Distance Vector keeps a routing table with the destination, next hop, and cost. It learns routes by exchanging distance information with its direct neighbors. Link State is different because each node collects information about the network topology and uses Dijkstra to calculate the shortest paths. Link State needs more information stored, and its updates are flooded through the network, while Distance Vector sends its updates only to neighbors. Distance Vector is simpler to start with, but handling failed links and routing loops can be harder. Link State can respond to topology changes once the updated information reaches the nodes, but maintaining the topology database and flooding LSAs makes the implementation more complicated.

2. Give an example explaining why an update from the current next hop must be processed even when it makes the route worse. What goes wrong if your node accepts only smaller costs?

Node 1 reaches Node 4 through Node 2 with a cost of 2. If the link between Nodes 2 and 4 fails, Node 2 might advertise a higher cost or a cost of 16 for Node 4. Node 1 needs to accept that update because Node 2 is the next hop it currently uses. If Node 1 only accepted smaller costs, it would keep using Node 2 even though that path no longer works. In my project, I tested this by removing the 2-4 link. Node 1 changed to the longer route through Node 3 with a cost of 3, then returned to Node 2 when the link was restored.

3. Use a small example to distinguish simple split horizon, poison reverse, and route poisoning after a failure. Why can poison reverse still permit loops involving three or more nodes?

Let's say Node A reaches Node D through Node B. With simple split horizon, Node A would leave out its route to D when sending an update to B. With poison reverse, it still includes D but advertises a cost of 16 to B. Route poisoning is when a route becomes unavailable after a failure and the node advertises it with a cost of 16 to let neighbors know. Poison reverse helps stop two nodes from using each other as the next hop for the same destination. But if three or more nodes could still have outdated information and temporarily point to one another in a loop. That is why my implementation also uses INFINITY = 16 and a TTL for forwarded packets.

4. Does your implementation necessarily produce symmetric forward and reverse paths? Explain how topology, equal-cost choices, and convergence affect the answer.

No, the forward and reverse paths do not always have to be the same. Each node builds its own routing table using the updates it receives from neighbors. If there are multiple routes with the same cost, different nodes could keep different next hops. During convergence, some nodes might also have newer routing information than others. In my line-topology test, the Ping and PingReply followed the same path in opposite directions because there was only one route. In a topology with multiple paths, that is not guaranteed even if the links are bidirectional.

5. What happens when a DV advertisement is lost or malformed? Explain how your periodic updates, validation, packetization, and expiration policy address these cases.

If a DV advertisement is lost, the neighbor doesn't receive that update, so it keeps its previously learned information until another update arrives or that information expires. My DV module sends periodic updates about every 10 seconds. If an entry is not refreshed for 5 update intervals, it can expire, and the route gets recalculated.

The receive handler also checks the packet type, destination, TTL, version, and number of entries before processing an advertisement. Entries have a bounded size, and the module sends larger routing tables across multiple packets instead of trying to fit everything into one. In the lossy test, I saw routes become unreachable and later recover when new updates were received.

6. What happens if a node repeatedly advertises and withdraws a route, or advertises a route but never forwards packets? Describe one possible mitigation and its limitations.

If a node keeps advertising and withdrawing the same route, other nodes might keep changing their routing tables. This can cause unstable routes, temporary packet loss, and more control traffic. One possible improvement would be to wait for a route to stay stable before switching back to it, although that could make recovery slower. If a node advertises a route but does not actually forward packets, other nodes could still choose it as their next hop and lose packets. A possible solution would be to track whether packets are successfully delivered or use link-quality information before trusting that route. That would also add more overhead, and it wouldn't always tell us whether the problem is a bad link or the forwarding node itself.